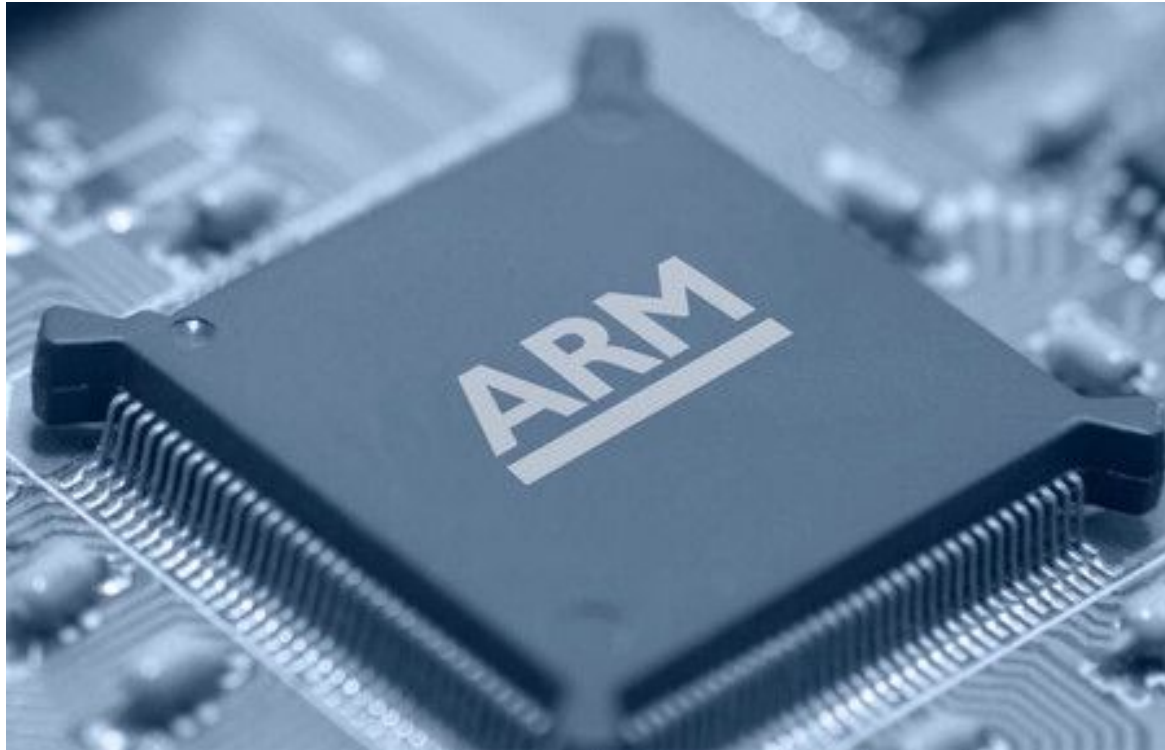






¿Por qué ARM?

- ❑ Arquitectura RISC pensada para simplicidad y eficiencia.
- ❑ Dominante en sistemas embebidos, móviles y SoC.
- ❑ Filosofía RISC: menos instrucciones → ciclo más corto → menor consumo.
- ❑ ARM v7-A (AArch32) será nuestro foco.

Historia de ARM

1990



ARM

2004



ARM

2018



arm

Diseño de procesadores

- No fabrican procesadores
- Utilizan software para simular el microcódigo que realizan las compuertas de silicio
- Simulación lógica
- Simulación eléctrica
- Simulación térmica

Ejemplo de lenguaje : VERILOG

Lenguaje de tipo HDL (Hardware Description Language)



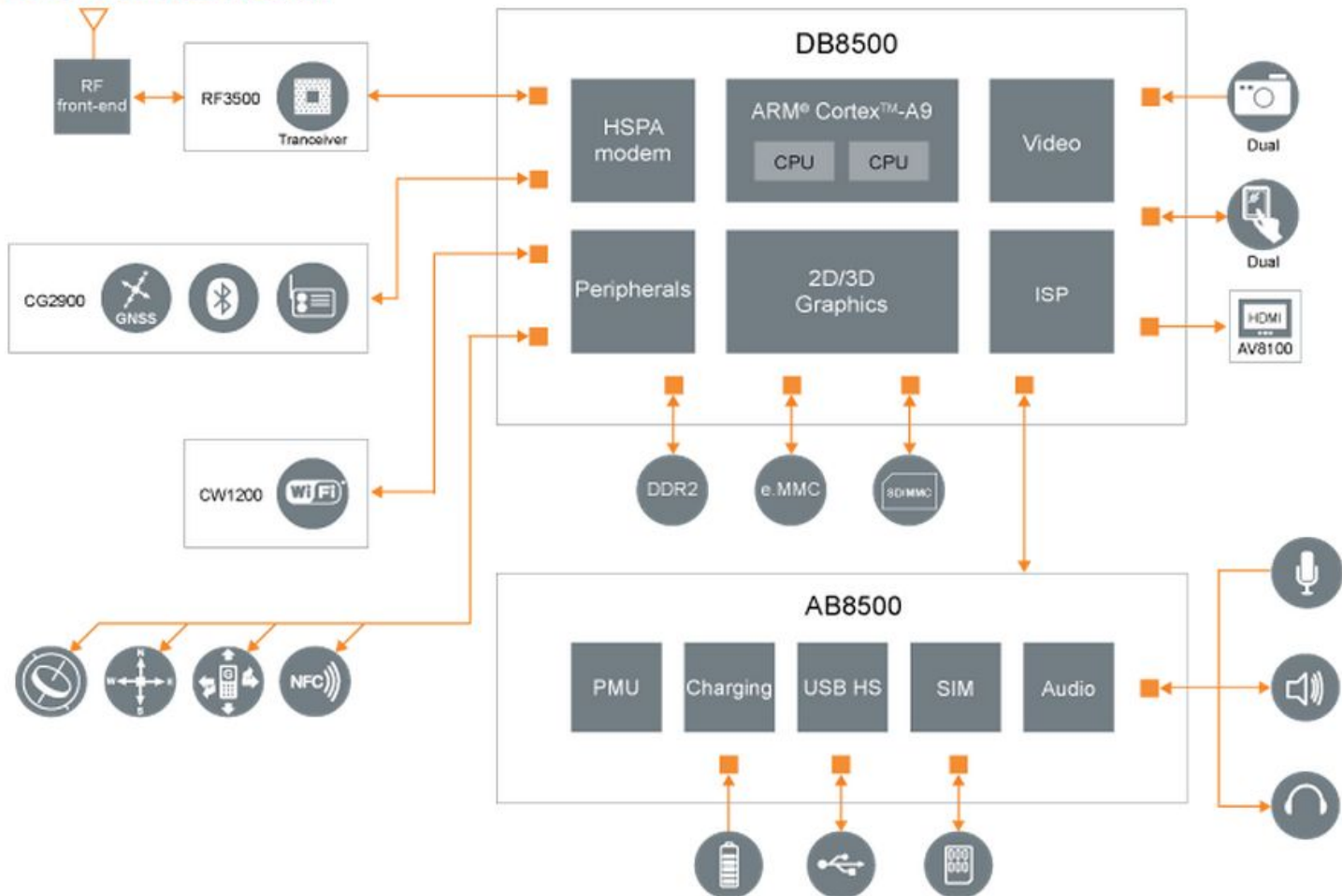
Systems on Chip (SoC)

- Un SoC está integrado por:
 - Procesador
 - Memorias (ROM, RAM, Flash)
 - Osciladores
 - Conversores A/D y D/A
 - Interfaces (USB, Ethernet, USART)

Systems on Chip (SoC)

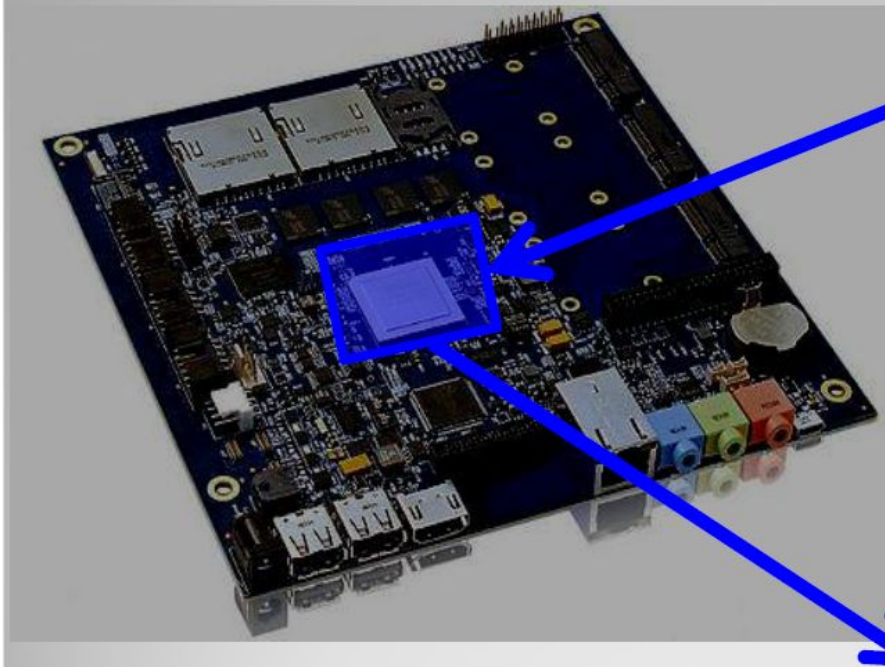
- Ejemplo: ST-Ericcson
- Novathor U8500

U8500 BLOCK DIAGRAM



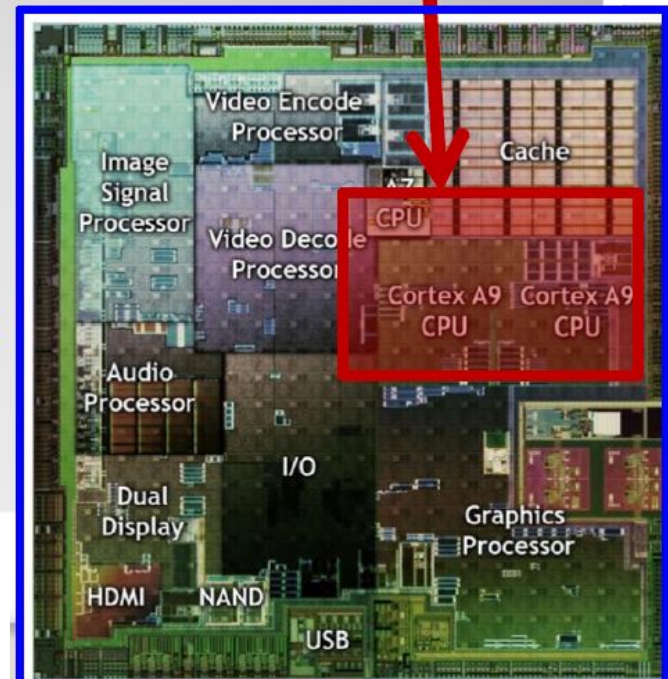
Systems on Chip (SoC)

ARM Embedded Processor



**Main CPU Chip
(SoC)**

**ARM Embedded
Processor**



Ecosistema ARM

- ❑ Perfiles:
 - A (Application) → smartphones, tablets, SBC.
 - R (Real Time) → control industrial, automotriz.
 - M (Microcontroller) → Arduino Due, STM32.
- ❑ SoC = CPU + GPU + controladores + memoria.
- ❑ Ejemplo: Cortex-A9 en Raspberry Pi o DE1-SoC.



ARM Cortex A9

- Arquitectura ARMv7-A
- Simple ó Multi Procesador (hasta 4)
- Varios niveles de control de consumo
- Basado en RISC
- Instrucciones ARM y Thumb
- 37 registros de 32 bits

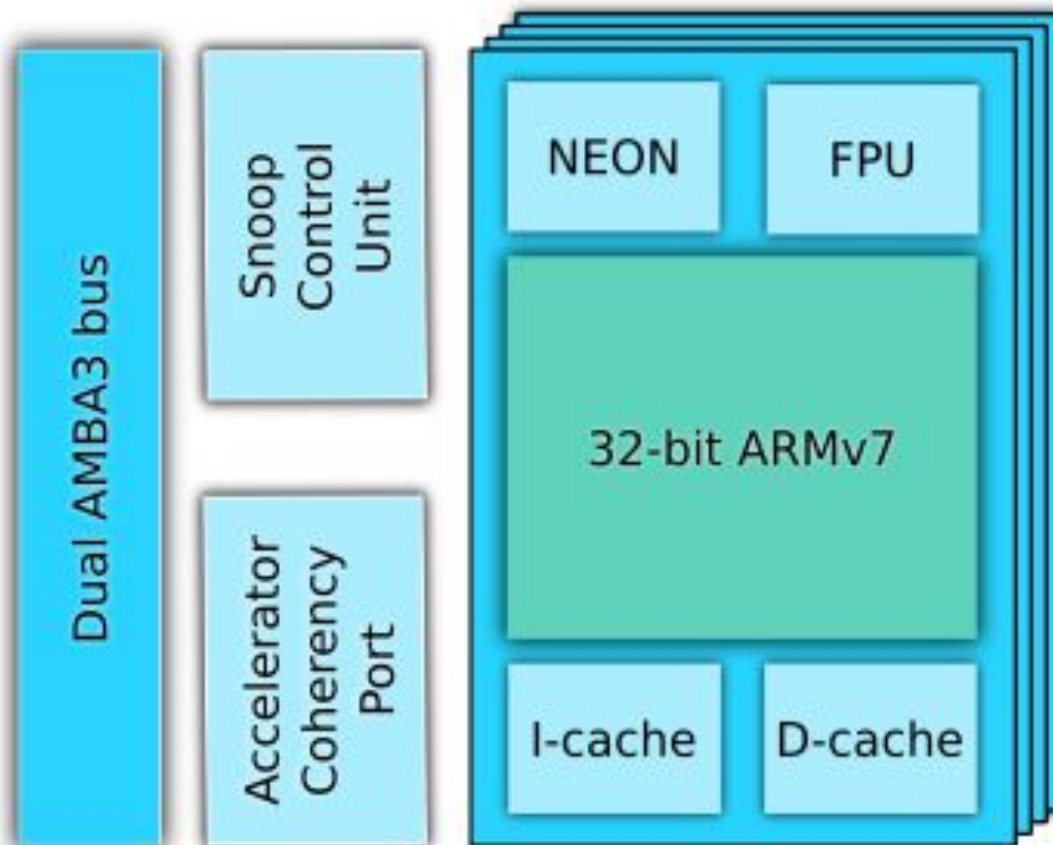


ARM Cortex A9

- Velocidad de clock entre 800MHz y 2 GHz
- Floating Point
- Controlador de caché L2
- Jazelle (Optimizador Java)
- DSP (Digital Signal Processing)

ARM Cortex A9

- Caché de Instrucciones/Datos de 16,32 o 64 KB
- Arquitectura **Harvard modificada**



ARM Cortex A9 SoC que lo usaban:

- Apple A5 (Ipad 2 y 3, Iphone 4s, Ipad mini)
 - L1 Cache 32 KB de instrucciones y 32 KB de datos
 - L2 Cache 1 MB
- OMAP4 (Motorola Razr, kindle Fire)
- Exynos 4 (Samsung Galaxy S3, Samsung Note)
 - Quad Core ARM-Cortex-A9
 - CPU 1.4-1.6 GHz

Modelos de programación

- ❑ 16 registros visibles (R0–R15).
- ❑ R13 = SP (stack), R14 = LR (link register), R15 = PC.
- ❑ CPSR contiene flags (N Z C V) + modo + bit T (Thumb).
- ❑ Arquitectura Load/Store: solo LDR/STR acceden a memoria.

Set de Registros

Current Visible Registers

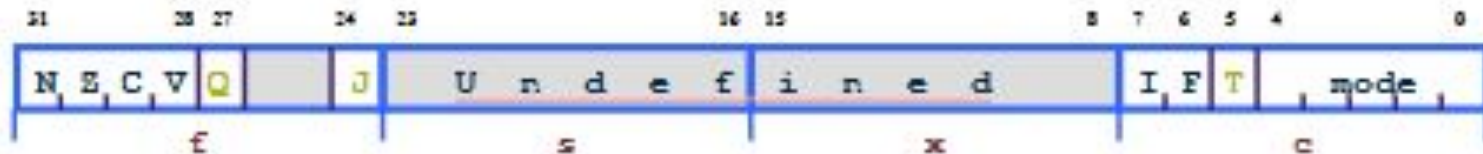
Abort Mode

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13(sp)
r14(lr)
r15(pc)
cpsr
spsr

Banked out Registers

User	FIQ	IRQ	SVC	Undef
	r8			
	r9			
	r10			
	r11			
	r12			
r13(sp)	r13(sp)	r13(sp)	r13(sp)	r13(sp)
r14(lr)	r14(lr)	r14(lr)	r14(lr)	r14(lr)
	spsr	spsr	spsr	spsr

Registros - Flags



■ Condition code flags

- N = **N**egative result from ALU
- Z = **Z**ero result from ALU
- C = ALU operation **C**arried out
- V = ALU operation **o**Verflowed

■ Sticky Overflow flag - Q flag

- Architecture 5TE/J only
- Indicates if saturation has occurred

■ J bit

- Architecture 5TEJ only
- J = 1: Processor in Jazelle state

■ Interrupt Disable bits.

- I = 1: Disables the IRQ.
- F = 1: Disables the FIQ. (Fast IRQ)

■ T Bit

- Architecture xT only
- T = 0: Processor in ARM state
- T = 1: Processor in Thumb state

■ Mode bits

- Specify the processor mode

Ejecución condicional

- ❑ Cada instrucción puede ejecutarse según flags.
- ❑ Sufijos EQ, NE, LT, GT, GE, LE.
- ❑ Reduce saltos → mejora pipeline.
- ❑ Ejemplo: `MOVLT r2, r1` // si $r1 < 0 \rightarrow r2 = r1$.



Modos y estados

- ❑ Modo ARM (32 bits) / Thumb (16 bits).
- ❑ Bit T en CPSR indica estado actual.
- ❑ Cambios de estado con BX / BLX.
- ❑ Thumb-2 combina instrucciones de 16 y 32 bits.

Convención AAPCS

- ❑ *ARM Architecture Procedure Call Standard*
- ❑ R0–R3 → parámetros.
- ❑ R0 → retorno.
- ❑ R4–R11 deben **preservarse** por callee.
- ❑ Stack descendente, alineado a palabra.

ARM vs x86

Aspecto	ARM	x86
<i>Filosofía</i>	RISC	CISC
<i>Instrucciones</i>	Cortas y uniformes	Largas y variables
<i>Consumo</i>	Bajo	Alto
<i>Pipeline</i>	Estático simple	Profundo con predicción



Qué es Thumb

- ❑ Subconjunto compacto de ARM (16 bits).
- ❑ 30 % menos tamaño de código.
- ❑ Ideal para microcontroladores y memorias reducidas.

Cambio de estado

BX	destino	@ cambia ARM↔Thumb
BLX	destino	@ llamada + cambio
.arm / .thumb		@ directivas del ensamblador

Bit T del CPSR cambia automáticamente



Ventajas de Thumb

- ☐ Mejor densidad de código.
- ☐ Ahorro energético por menor ancho de bus.
- ☐ Ligeras limitaciones en registros por instrucción.

Ejemplo comparativo

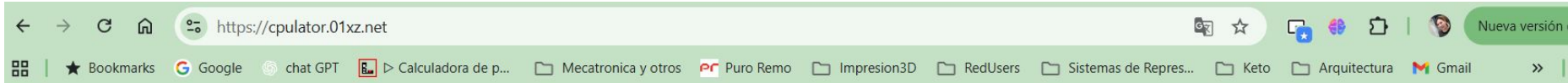
Operación	ARM (32 b)	Thumb (16 b)
Suma	<code>ADD r0, r1, r2</code> <code>; r0 = r1 + r2</code>	<code>ADD r0, #2</code>
Salto	<code>BNE loop</code>	<code>BNE loop</code> (mismo significado)



CPUlator

- ❑ <https://cpulator.01xz.net/>
- ❑ Elegir ARMv7 Generic (32-bit).
- ❑ Pestañas: Registers / Memory / Disassembly / Terminal.
- ❑ Atajos: F5 = Compile & Load, F6 = Step, Ctrl+Enter = Run.

CPUlator



CPUlator Computer System Simulator

CPUlator is a Nios II, ARMv7, MIPS, and RISC-V RV32 simulator of a computer system (processor and I/O devices) and debugger that runs in a modern web browser. It is designed as a tool for learning assembly-language programming and computer organization.

To start using CPUlator now, choose a computer system to simulate, then follow the link.

To learn more, try a sample program in the simulator (Help → Sample programs), or see the [documentation](#).

Choose a system to simulate

Architecture

Any
Nios II
ARMv7
MIPS32r5
MIPS32r6
RISC-V RV32

System

ARMv7 generic
ARMv7 DE1-SoC
ARMv7 DE1-SoC (v16.1)
Nios II generic
Nios II DE1-SoC
Nios II DE1-SoC (v16.1)
Nios II DE2-115

<https://cpulator.01xz.net/?sys=arm>

Go

CPUlator

The screenshot displays the CPUlator IDE interface. At the top, a toolbar includes buttons for **Stopped**, **Step Into** (F2), **Step Over** (Ctrl-F2), **Step Out** (Shift-F2), **Continue** (F3), **Stop** (F4), **Restart** (Ctrl-R), **Reload** (Ctrl-Shift-L), **File**, and **Help**.

The left sidebar contains several panels:

- Registers**: A table showing the state of 16 registers (r0-r15). All registers currently contain 00000000. A **Refresh** button is located above the table.
- Call stack**, **Trace**, **Breakpoints**, **Watchpoints**, **Symbols**, and **Counters**: These panels are currently collapsed.
- Settings**: A section for configuring the number display options, with **Size** set to **Word** and **Format** set to **Hexadecimal**.

The main **Editor (Ctrl-E)** window shows a C program named **untitled.s** with the following code:

```
1 .global _start
2 _start:
3
4 |
```

Below the editor, a toolbar provides quick access to **Editor (Ctrl-E)**, **Disassembly (Ctrl-D)**, and **Memory (Ctrl-M)**.

The bottom **Messages** panel displays the following message:

CPUlator has started with system **ARMv7 generic** containing a **ARMv7** processor.

A **Clear** button is located to the right of the message.

CPUlator

Stopped

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File ▾

Help ▾

Registers

r0	00000000	
r1	00000000	
r2	00000000	
r3	00000000	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000000	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

Registers Call stack Trace
Breakpoints Watchpoints Symbols
Counters

Settings

Number Display Options

Size: Word ▾

Format: Hexadecimal ▾

Memory words per row: 4 ▾

Messages

Editor (Ctrl-E)

Compile and Load (F5)

Language: ARMv7 ▾

untitled.s [changed since save] [changed since compile]

```
1 .text
2     .global _start
3     .arch armv7-a
4     .arm
5 _start:
6     MOV r0,#5
7     MOV r1,#7
8     ADDS r2,r0,r1
9     SUBS r3,r2,#20
10 loop: B loop
11
12
13
```

Editor (Ctrl-E) Disassembly (Ctrl-D) Memory (Ctrl-M)

CPUlator has started with system **ARMv7 generic** containing a **ARMv7** processor.

CPUlator

Stopped

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File ▾

Help ▾

Registers

r0	00000000	
r1	00000000	
r2	00000000	
r3	00000000	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000000	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

RegistersCall stackTraceBreakpointsWatchpointsSymbolsCounters

Settings

Number Display Options

Size: Word ▾

Format: Hexadecimal ▾

Memory words per row: 4 ▾

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 ▾ Refresh

Address	Opcode	Disassembly
		1 .text
		2 .global _start
		3 .arch armv7-a
		4 .arm
		5 _start:
		6 _start:
00000000	e3a00005	6 mov r0, #5 ; 0x5
00000004	e3a01007	7 mov r1, #7 ; 0x7
00000008	e0902001	8 adds r2, r0, r1
0000000c	e2523014	9 subs r3, r2, #20 ; 0x14
		10 loop: B loop
		loop:
00000010	eaaffffe	b 0x10 (0x10: loop)
00000014	00000000	andeq r0, r0, r0
		_end:
00000018	aaaaaaaa	bge 0xfeaaaaac8
0000001c	aaaaaaaa	bge 0xfeaaaacc
00000020	aaaaaaaa	bge 0xfeaaaad0
00000024	aaaaaaaa	bge 0xfeaaaad4
00000028	aaaaaaaa	bge 0xfeaaaad8
0000002c	aaaaaaaa	bge 0xfeaaaadc
00000030	aaaaaaaa	bge 0xfeaaaade
00000034	aaaaaaaa	bge 0xfeaaaad4
00000038	aaaaaaaa	bge 0xfeaaaad8
0000003c	aaaaaaaa	bge 0xfeaaaadec

Editor (Ctrl-E)Disassembly (Ctrl-D)Memory (Ctrl-M)

Messages

Compiling...

Code and data loaded from ELF executable into memory. Total size is 24 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asmwdSzhk.s.o work/asmwdSzhk.s

Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmwdSzhk.s.elf work/asmwdSzhk.s.o

Compile succeeded.

Modos de direccionamiento

- ❑ Inmediato → MOV r0,#5
- ❑ Base + offset → LDR r1,[r0,#4]
 - Contenido de [r0+4]
- ❑ Indexado → LDR r1,[r0,r2,LSL #2]
 - LSL = Logical Shift Left
 - dirección = $r0 + (r2 \ll 2)$
 - equivale a multiplicar por 4
- ❑ Pre/post-incremento → LDR r1,[r0],#4
 - Primero lee contenido de r0
 - luego suma 4 a r0
 - para recorrer vectores

Alineación y Endianness

- ❑ ARM es **bi-endian**: puede trabajar en *big* o *little endian*.
- ❑ En **Cortex-A y CPULator**, el modo por defecto es *little-endian*.
- ❑ Las palabras de 32 bits deben comenzar en direcciones múltiplo de 4.
- ❑ El orden de los **bytes** en memoria no afecta el orden de los **bits** dentro de cada byte.



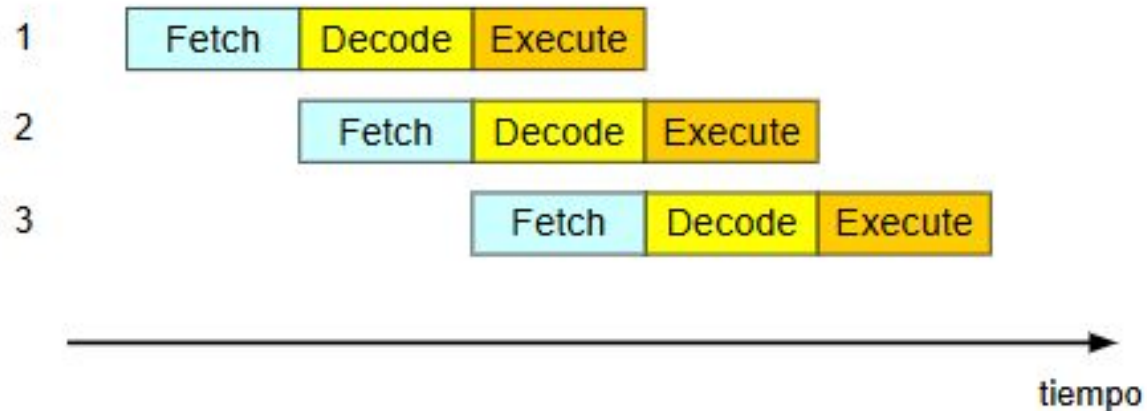
Ejemplo de acceso a arreglo

LDR r0, =arr

LDR r1, [r0, #8] @ lee 3er elemento (2×4)

Pipeline en ARMv7

- ❑ 3 etapas: Fetch → Decode → Execute.
- ❑ Un salto rompe la prefetch pipeline (“burbuja”).
- ❑ Ejecución condicional reduce saltos.



Predicción y Thumb-2

- ❑ Thumb-2 = mezcla de 16/32 b + predicción simplificada.
- ❑ Mayor rendimiento sin sacrificar tamaño.

```
If ( a==b)  
{ a++; } else { a--; }
```



```
Cmp R1,R2 /* se asume R1 vale a y R2 b */  
Addeq R1,#1 /* suma 1 a R1 */  
Subne R1,#1 /* resta si no es igual */
```



Pipeline vs Consumo

- ❑ Diseño simple → menos transistores.
- ❑ ARM logra rendimiento por MHz similar a x86 con menor energía.

CPUlator ejemplo 1

```
.text
.global _start
.arch armv7-a
.arm
_start:
    MOV r0,#5
    MOV r1,#7
    ADDS r2,r0,r1
    SUBS r3,r2,#20
loop:  B loop
```

Objetivo: observar flags N,Z,C,V en CPSR.

Dato extra: SVC quiere decir “Modo supervisor”

CPUlator ejemplo 2

```
.data
arr: .word 3,4,7,-2,0
N:   .word 5
sum: .word 0
    .text
    .global _start
    .arm
_start:
    LDR r0,=arr
    LDR r1,=N
    LDR r1,[r1]
    MOV r2,#0
    MOV r3,#0
```

```
loop:
    CMP r3,r1
    BEQ end
    LDR r4,[r0,r3,LSL #2]
    ADD r2,r2,r4
    ADD r3,r3,#1
    B loop
end:
    LDR r5,=sum
    STR r2,[r5]
stop: B stop
```

Objetivo: sumar elementos de un arreglo.



Redondeando

- ❑ ARM combina eficiencia, simplicidad y versatilidad.
- ❑ Es la base de la computación móvil y embebida moderna.
- ❑ CPUlator nos permite explorar toda la arquitectura ARMv7-A.
- ❑ **Fin de la cursada – ¡Gracias por acompañar todo el recorrido!**